

AFL Assistant — Executive Report

Product goal. Deliver a domain-locked AFL chat and prediction assistant that answers grounded AFL questions, retrieves player/team statistics, and provides probabilistic match predictions through a LangGraph workflow.

Architecture. The system routes requests between factual chat, retrieval tools, and prediction models. LangGraph manages routing and state; retrieval tools provide AFL evidence; prediction models return probabilities; FastAPI exposes a chat endpoint; an optional Streamlit interface provides a simple demo surface. Structured logs capture query, intent, tools, latency, and token usage.

Evaluation. The combined suite contains 32 cases: 8 factual Q&A, 6 prediction-sanity, 10 scope-guardrail, and 8 conversational-coherence cases. The prepared evaluation summary contains 29 passes out of 32 (90.6%). Factual Q&A and scope guardrails are 100%; prediction sanity is 83.3%; conversational coherence is 75.0%. The weakest category is conversational coherence, so the concrete improvement is stronger structured conversation state plus additional multi-turn regression tests.

System hardening. Prediction responses use the consistent phrase “predicted probability, not a certainty.” Tool calls should use explicit timeouts and return controlled error messages. Three prompt-injection style tests are included to verify that attempts to override the AFL-only scope are blocked. Repeated off-topic probing should be logged and rate-limited.

Known limitations. Data recency depends on the latest supplied AFL data. Prediction accuracy has a ceiling determined by feature quality, sample size, injuries/selection information, and match randomness. Guardrails can also face ambiguous multi-turn prompts or adversarial wording. The ladder-position benchmark should be measured on exactly the same held-out matches before drawing conclusions about model performance.

Recommended next steps. Automate ingestion after each completed round, run the regression suite, monitor prediction calibration and accuracy, refresh the feature table, and retrain only when the agreed trigger is met. Improve conversation-state handling and expand adversarial guardrail tests.

Monitoring & Maintenance Checklist

Metric	Example alert threshold	Cadence	Action
p95 response latency	> 3 seconds	Daily	Inspect slow nodes/tools
Tool error rate	> 5%	Daily	Check API/data source failures
Off-topic leak rate	> 1%	Weekly	Run guardrail regression and tighten routing
Prediction accuracy drift	Drop > 5 percentage points	After each round	Evaluate/retrain candidate
Prediction calibration/Brier	Material deterioration vs baseline	After each round	Recalibrate or retrain
Prompt-injection pass-through	> 10 critical cases	Weekly	Block/regression test immediately

Weekly refresh loop: (1) ingest completed round results; (2) append match/player outcomes to the feature table; (3) recompute rolling and season features; (4) run data-quality checks; (5) score the current model on the newest completed matches; (6) compare against the ladder-position benchmark and previous model; (7) retrain when the predefined drift/performance trigger is reached; (8) run the 32-case regression suite; (9) publish the refreshed model only if guardrails, calibration, and accuracy checks pass.